



Note

Exact weighted vertex coloring via branch-and-price

Fabio Furini, Enrico Malaguti*

Dipartimento di Elettronica, Informatica e Sistemistica, University of Bologna, Viale Risorgimento, 2-40136-Bologna, Italy

ARTICLE INFO

Article history:

Received 24 November 2011

Received in revised form 6 February 2012

Accepted 20 March 2012

Available online 16 April 2012

Keywords:

Vertex coloring

Column generation

Branch-and-price

Computational experiments

ABSTRACT

We consider the Weighted Vertex Coloring Problem (WVCP), in which a positive weight is associated to each vertex of a graph. In WVCP, one is required to assign a color to each vertex in such a way that colors on adjacent vertices are different, and the objective is to minimize the sum of the costs of the colors used, where the cost of each color is given by the maximum weight of the vertices assigned to that color. This NP-hard problem arises in practical scheduling applications, where it is also known as Scheduling on a Batch Machine with Job Compatibilities. We propose the first exact algorithm for the problem, which is based on column generation and branch-and-price. Computational results on a large set of instances from the literature are reported, showing excellent performance when compared with the best heuristic algorithms from the literature.

© 2012 Elsevier B.V. All rights reserved.

1. Introduction

Given an undirected graph $G = (V, E)$ (with V denoting the vertex set, and E the edge set), we consider the *Weighted Vertex Coloring Problem* (WVCP), in which a positive weight is associated to each vertex of the graph. In WVCP, one is required to assign a color to each vertex in such a way that adjacent vertices receive different colors, and the objective is to minimize the sum of the costs of the colors used, where the cost of each color is given by the maximum weight of the vertices assigned to that color (whereas in the traditional Vertex Coloring Problem (VCP) the aim is to minimize the number of colors used). Since an instance of VCP can be transformed to an instance of WVCP by defining a weight of 1 for each vertex, the problem is NP-hard (see Garey and Johnson [1] for complexity results on VCP, and Malaguti [2] and Malaguti and Toth [3] for other NP-hard generalizations of the VCP).

In the following we let $n = |V|$ and $m = |E|$. A subset of V is called an *independent set* if no two adjacent vertices belong to it. Note that each coloring of a graph is a partition of the vertex set into independent sets; we will call these sets *color classes*. An independent set is maximal if no vertex can be added still having an independent set. We denote the weight of a vertex i by w_i ; given an independent set $S \subseteq V$ of weighted vertices, its *cost* w corresponds to the maximum weight of its vertices.

De Werra et al. [4] analyze some properties of the optimal solutions and discuss complexity and approximability results for this problem; Escoffier et al. [5] continue the investigation of the complexity and the approximability of WVCP. The problem is NP-hard even for bipartite graphs, but in this case it can be solved in polynomial time within an approximation ratio of $4/3$ (the bound is tight). Boudhar and Finke [6] study its complexity for several classes of graphs. These results are extended to different classes of graphs by Finke et al. [7]. The problem remains NP-hard for general graphs when the cardinality of the color classes is bounded above by a constant k , for $k \geq 3$; in the unbounded case, it can be solved in polynomial time for circular, chordal, comparability and interval graphs.

WVCP is also known in scheduling as *Scheduling on a Batch Machine with Job Compatibilities*, defined by Boudhar and Finke [6] as the problem of minimizing the makespan on a single batch processing machine, in which there are jobs that

* Corresponding author. Tel.: +39 0512093942; fax: +39 0512093073.

E-mail addresses: fabio.furini@unibo.it (F. Furini), enrico.malaguti@unibo.it (E. Malaguti).

cannot be processed simultaneously in the same batch. The processing time of a batch is given by the processing time of the longest jobs in the batch; the capacity of the batch processing machine can be finite or infinite (this latter case is equivalent to the WVCP). In addition, the problem generalizes the *Matrix Decomposition Problem in Time Division Multiple Access Traffic Assignment*, where a traffic matrix has to be decomposed into mode matrices (having no more than one non zero element present in each row and column), and the cost (transmission time) of each mode matrix equals the maximum of its non zero elements. Ribeiro et al. [8] propose an exact approach for this matrix decomposition problem based on column generation, while Prais and Ribeiro [9] have proposed a heuristic approach based on a Greedy Randomized Adaptive Search Procedure (GRASP).

Concerning computational approaches to the more general WVCP, the only contributions we are aware of are heuristic algorithms. Malaguti et al. [10] propose a two-phase algorithm, where in the first phase a large number of independent sets is heuristically produced, and in the second phase the set covering instance associated with these sets (see Section 1.1) is heuristically solved through a Lagrangian heuristic algorithm (CFT by Caprara et al. [11]). In [10], the authors also propose and compare three integer linear programming (ILP) models for the problems, and use the linear relaxation of one of these models to compute a lower bound for the problem. Recently, Oliveira et al. [12] proposed a heuristic algorithm based on Variable Neighborhood Descent and backtracking, which is tested on the same instances considered in [10].

The paper is organized as follows: in the next Section we review an ILP model, involving an exponential number of variables, which is solved through branch-and-price (algorithm outlined in Section 2). Computational experiments on a set of instances previously considered in the literature are reported in Section 3. Concluding remarks are made in Section 4.

1.1. ILP models for WVCP

In [10] the authors propose a compact model for the WVCP, based on the idea of having each color class initialized by a “special” vertex, whose weight determines the cost of the class. They also prove that this model dominates a traditional descriptive model in terms of value of the linear relaxation, which is used as a lower bound. Finally, they extend the set covering formulation, proposed for the classical VCP by Mehrotra and Trick [13], to the WVCP. However, no computational results for this formulation are reported in the paper. We review in the following the set covering formulation of WVCP, that we solve by branch-and-price, as described in the next section.

Let W denote the set of weights that appear in V , i.e., $W = \{w: \exists i \in V, w_i = w\}$, and $\mathcal{S}_w$ the family of all the independent sets of G having the heaviest vertex of weight w . We introduce, for each independent set $S \in \mathcal{S}_w$ and for each $w \in W$, a binary variable x_S which can take the value 1 when all the vertices of S receive the same color, and has the value 0 when at least two vertices in S receive different colors. For each $S \in \mathcal{S}_w$, the associated cost is w , which represents the cost to be paid if independent set S is assigned a color. We obtain the following ILP model for WVCP:

$$\min \sum_{w \in W} \sum_{S \in \mathcal{S}_w} w x_S \quad (1)$$

$$\sum_{w \in W} \sum_{S \in \mathcal{S}_w: i \in S} x_S \geq 1, \quad i \in V, \quad (2)$$

$$x_S \in \{0, 1\}, \quad w \in W, S \in \mathcal{S}_w. \quad (3)$$

Objective function (1) minimizes the sum of the costs of the selected independent sets (each of them will correspond to a color class), while constraints (2) impose that, for each vertex, at least one independent set containing the vertex is selected. The objective function forces the x_S variables that are not needed in order to satisfy constraints (2) to equal 0 because, e.g., there is a variable $x_{S'} = 1$ with $S \subset S'$. Although this formulation may produce solutions which assign more than one color to some vertices, it is straightforward to obtain equivalent solutions where each vertex receives exactly one color.

2. Branch-and-price

In this section we describe the main components of our branch-and-price algorithm. Model (1)–(3) has exponentially many binary variables (corresponding to the independent sets of G) but, by using column generation techniques, we can restrict ourselves to the subset of variables that we need for optimality. By relaxing the integrality constraint (3) to:

$$x_S \geq 0 \quad w \in W, S \in \mathcal{S}_w \quad (4)$$

we obtain the so-called *master problem*, whose rounded-up optimal solution value is the lower bound on the optimal solution value of (1)–(3) used during the branch-and-price. To solve the master problem, we iteratively consider a subfamily of the family $\mathcal{S}_w$, $w \in W$, of all the independent sets of G , i.e., at each iteration, we have a *restricted master problem*. By solving the restricted master problem to optimality we obtain the values (denoted as *profits* in the following) π_i^* , $i \in V$, of the dual variables associated with constraints (2). Dual constraints read:

$$\sum_{i \in S} \pi_i \leq w \quad w \in W, S \in \mathcal{S}_w. \quad (5)$$

The variables (independent sets) to be added to the restricted master problem correspond to violated dual constraints. To detect such violated constraints, we need to solve one *slave problem* for each $w \in W$. The generic slave problem is to find an independent set S^* for which (5) is violated, that is, an independent set of total profit larger than w in $G_w = (V_w, E_w)$, which is the subgraph of G induced by the subset of vertices $V_w = \{i : w_i \leq w\}$. Thus, for each $w \in W$, the slave problem can be encoded by the following ILP, where binary variables y_i , ($i \in V_w$), take the value 1 when vertex i belongs to S^* and 0 otherwise:

$$\max \sum_{i \in V_w} \pi_i^* y_i \quad (6)$$

$$y_i + y_j \leq 1 \quad (i, j) \in E_w \quad (7)$$

$$y_i \in \{0, 1\} \quad i \in V_w. \quad (8)$$

Model (6)–(8) defines a *Maximum Weighted Independent Set Problem* (MWISP), with profits (weights) π^* . This problem is NP-hard on general graphs (see Garey and Johnson [1]). If the optimal solution of MWISP, restricted to vertices $i \in V_w$, has a value greater than w , then we have found an independent set with negative reduced cost, we add the corresponding column to the restricted master problem and iterate. Otherwise, the current restricted master problem contains all the columns corresponding to an optimal solution, and hence model (1), (2) and (4) is optimally solved.

2.1. Solution of the slave problem

Solving to optimality several MWISPs in sequence may be a time-consuming task. However, we do not need to solve the problem to optimality when new columns with negative reduced cost are available. Optimality of the slave problems is required only to certify that such columns do not exist.

Thus, during the column generation, new columns are produced through the tabu search algorithm for the MWISP proposed by Malaguti et al. in [14] and used as a subroutine in their branch-and-price algorithm for the VCP. The tabu search algorithm starts by computing a maximal independent set in a greedy fashion, and then performs a series of one and two swaps, in order to possibly improve the value of the independent set. When looking for a weighted independent set of profit at least w , a threshold βw with $\beta > 1$ is used, and the algorithm is stopped as soon as an independent set with profit larger than the threshold is found. Although it would be enough to generate an independent set with profit larger than w , generating columns with even larger profit is preferable, because experimentally the method converges in a smaller number of iterations. Thus, when using a fast algorithm during the column generation, there is a computational advantage in spending a little more time in producing better columns. Clearly, the value of β has to be experimentally tuned. We try to generate one column through tabu search for each $w \in W$ and, when we succeed for one value of w , we do also consider all other values in W before re-optimizing the master.

If no column with negative reduced cost is generated by the tabu search, we resort to an exact Branch-and-Bound to generate such a column, if any exists. We use the Branch-and-Bound for the MWISP recently proposed by Held et al. [15] (publicly available at <https://code.google.com/p/exactcolors/>) and used as a subroutine in their branch-and-price algorithm for the VCP. This algorithm can also be stopped as soon as an independent set with profit larger than a specified threshold is found. In both the heuristic and exact generation, we set a threshold βw with $\beta = 1.1$. When using this exact method for the MWISP, we re-optimize the restricted master problem every time a new column with negative reduced cost is generated. Finally, if such a column does not exist, we have optimally solved the master (1), (2) and (4).

When the optimal solution of the master problem is integer, we have the optimal solution to the WVCP. Otherwise, the rounded-up value of the master optimal solution is a lower bound on the solution value of WVCP, and we need to embed column generation in a branch-and-price scheme to find the optimal *integer* solution.

2.2. Branching

When branching, we choose the variable that has the largest fractional part in the current solution of the master problem as the branching variable, and implement the branching rule proposed by Zykov [16] for VCP. The idea of this rule is to select two vertices i and j such that $(i, j) \notin E$, and then consider the two subproblems which are obtained (1) by collapsing i and j into a single vertex k , with $(k, h) \in E$ for every vertex h for which $(i, h) \in E$ or $(j, h) \in E$, and (2) by adding an edge between i and j , i.e., by setting $E = E \cup (i, j)$. The branching scheme is robust in the sense that, after branching, the problem to be solved is still a WVCP; in case (1) the graph has one node less, in case (2) the graph has one additional edge. The obtained search tree is explored according to a *depth first* strategy.

Our preliminary computational experiments showed that, in order to help the algorithm to find feasible solutions of good quality, it is preferable to collapse “heavy” nodes having similar weights (possibly the same weight). The rationale for this choice is the following: when we collapse two nodes i and j in a new node k , this is equivalent to assigning the same color to i and j in a constructive algorithm. Since the cost of a color class is given by the “heaviest” node, it makes sense to try to put “heavy” nodes in the same color class. In selecting the two vertices i and j for branching, first we determine the most fractional variable, say x_{S1} , with $S1 \in \mathcal{S}_w$. We choose i as the first row covered by column $S1$ such that $w_i = w$ (i.e., we

Table 1

Results on instances derived from DIMACS instances.

Instance	<i>n</i>	<i>m</i>	B&P							M–M–T [10]				O–N–U [12]	
			<i>bd</i>	Time	<i>z</i>	Time	Gap	Cols	Nodes	<i>bd</i>	<i>z</i>	Time	Gap	<i>z</i>	Time
DSJC125_1g	125	736	21	33	27	<i>tl</i>	28.6	72,339	2,236	19	24	152	26.3	24	47
DSJC125_1gb	125	736	82	49	96	<i>tl</i>	17.1	53,180	1,231	74	95	170	28.4	92	53
DSJC125_5g	125	3891	68	2	82	<i>tl</i>	20.6	9,410	178,938	–	76	180		76	68
DSJC125_5gb	125	3891	230	32	273	<i>tl</i>	18.7	9,443	69,119	–	251	182		252	96
DSJC125_9g	125	6961	168	0	169	2	0.0	570	249	152	169	162	11.2	170	66
DSJC125_9gb	125	6961	602	4	604	5	0.0	691	26	568	605	237	6.5	606	58
GEOM30b	30	111	12	0	12	0	0.0	71	1	12	12	0	0.0	12	0
GEOM40b	40	197	16	0	16	0	0.0	113	1	16	16	1	0.0	16	0
GEOM50b	50	299	18	0	18	0	0.0	153	3	18	18	0	0.0	18	0
GEOM60b	60	426	23	0	23	0	0.0	340	5	23	23	0	0.0	23	0
GEOM70	70	337	47	0	47	1	0.0	685	8	47	47	96	0.0	47	1
GEOM70a	70	529	73	0	73	1	0.0	527	9	73	73	3	0.0	73	2
GEOM70b	70	558	24	0	24	0	0.0	275	2	24	24	6	0.0	24	1
GEOM80	80	429	66	0	66	2	0.0	959	27	66	66	0	0.0	66	0
GEOM80a	80	692	76	0	76	1	0.0	742	8	76	76	102	0.0	76	1
GEOM80b	80	743	27	0	27	1	0.0	518	3	27	27	90	0.0	27	14
GEOM90	90	531	61	1	61	4	0.0	1,170	20	61	61	166	0.0	61	38
GEOM90a	90	879	73	0	73	1	0.0	1,046	7	73	73	157	0.0	74	49
GEOM90b	90	950	30	0	30	1	0.0	495	9	30	30	11	0.0	30	2
GEOM100	100	647	65	2	65	12	0.0	2,278	25	65	65	131	0.0	65	15
GEOM100a	100	1092	89	0	89	2	0.0	1,115	10	89	89	112	0.0	89	9
GEOM100b	100	1150	32	0	32	1	0.0	663	8	32	32	15	0.0	32	4
GEOM110	110	748	68	1	69	<i>tl</i>	1.5	51,238	29,300	66	69	172	4.5	68	10
GEOM110a	110	1317	97	2	97	7	0.0	1,808	17	97	97	111	0.0	97	96
GEOM110b	110	1366	37	1	37	2	0.0	969	22	37	37	5	0.0	37	28
GEOM120	120	893	72	2	72	12	0.0	2,552	34	72	72	157	0.0	72	11
GEOM120a	120	1554	105	2	105	11	0.0	2,192	23	105	105	136	0.0	105	23
GEOM120b	120	1611	35	0	35	1	0.0	1,079	5	35	35	14	0.0	36	47
R50_1g	50	108	14	0	14	1	0.0	298	9	14	14	0	0.0	14	0
R50_1gb	50	108	53	0	53	0	0.0	291	5	52	53	95	1.9	53	0
R50_5g	50	612	36	1	37	2	0.0	720	287	35	37	167	5.7	37	11
R50_5gb	50	612	131	1	135	22	0.0	1,752	1,569	126	137	145	8.7	137	18
R50_9g	50	1092	74	0	74	0	0.0	103	1	73	74	36	1.4	74	1
R50_9gb	50	1092	262	0	262	0	0.0	192	3	257	262	33	1.9	262	5
R75_1g	70	251	18	1	18	12	0.0	3,682	117	17	19	154	11.8	19	10
R75_1gb	75	251	67	3	70	447	0.0	26,327	1,204	63	72	166	14.3	73	29
R75_5g	75	1407	49	0	51	2136	0.0	9,185	112,287	43	53	172	23.3	53	57
R75_5gb	75	1407	179	1	187	<i>tl</i>	4.5	9,718	109,969	160	190	173	18.8	191	54
R75_9g	75	2513	109	0	110	0	0.0	233	4	108	110	79	1.9	110	1
R75_9gb	75	2513	396	1	396	1	0.0	408	1	393	399	50	1.5	397	21
R100_1g	100	509	20	7	25	<i>tl</i>	25.0	67,247	27,848	18	22	155	22.2	22	22
R100_1gb	100	509	76	19	90	<i>tl</i>	18.4	65,990	6,678	70	84	171	20.0	84	79
R100_5g	100	2456	56	1	65	<i>tl</i>	16.1	12,205	137,308	48	62	179	29.2	62	105
R100_5gb	100	2456	212	2	233	<i>tl</i>	9.9	12,100	76,915	182	234	179	28.6	232	145
R100_9g	100	4438	141	0	141	0	0.0	326	41	138	142	123	2.9	143	75
R100_9gb	100	4438	517	2	518	5	0.0	617	190	499	520	127	4.2	518	69

choose a row of maximum weight in column S_1). Then we determine another column S_2 (in the base of the current solution of the master problem) that covers row i , and such that $S_2 \in \mathcal{S}_w$ (i.e., S_2 has the same cost w). If such a column does not exist, we look for a basic column $S_2 \in \mathcal{S}_v$, with v the largest weight smaller than w or, if such a column does not exist, v the smallest weight larger than w . Finally we find row j of maximum weight such that only one of the columns S_1 or S_2 covers it.

3. Computational experiments

In this section we report the results for the 46 weighted VCP instances (see [17]) obtained by adding weights to vertices of the original DIMACS benchmark graph instances [18], and considered in [10,12] (Table 1); 35 traffic decomposition matrix instances from [8], and considered in [10,9], and 30 additional traffic decomposition matrix instances from [9] and considered in [10,9] (Tables 2 and 3).

To allow a meaningful – although approximate – comparison on results obtained with different machines, a benchmark program (dfmax), together with a benchmark instance (r500.5), are available at <http://mat.gsia.cmu.edu/COLOR03/>. Computing times obtained on different machines can be scaled with respect to the performance obtained on this program. All results of our algorithms were obtained on one core of a core i5–650 at 3.20 GHz with 8 Gb RAM under the Linux operating system, which spent 5.82 s *user time* to solve the benchmark instance. In the experiments, we used CPLEX12.2 as the linear programming solver.

Table 2Matrix decomposition p instances.

Instance	n	m	B&P						M-M-T [10]				P-R [9]	
			bd	Time	z	Time	Cols	Nodes	bd	z	Time	Gap	z	Time
$p06$	16	38	565	0	565	0	37	1	565	565	0	0.0	565	1
$p07$	24	92	3771	0	3771	0	105	1	3771	3771	0	0.0	3771	4
$p08$	24	92	4049	0	4049	0	109	1	4049	4049	1	0.0	4049	1
$p09$	25	100	3388	0	3388	0	142	2	3388	3388	0	0.0	3388	3
$p10$	16	32	3983	0	3983	0	56	3	3983	3983	0	0.0	3983	4
$p11$	18	48	3380	0	3380	0	69	2	3380	3380	0	0.0	3380	4
$p12$	26	90	657	0	657	0	95	1	657	657	0	0.0	657	3
$p13$	34	160	3220	1	3220	1	224	1	3220	3220	8.7	0	3230	7
$p14$	31	110	3157	0	3157	0	170	4	3157	3157	0	0.0	3157	10
$p15$	34	136	341	0	341	0	212	1	341	341	0	0.0	341	4
$p16$	34	134	2343	0	2343	0	251	1	2343	2343	0	0.0	2343	14
$p17$	37	161	3281	0	3281	0	270	1	3281	3281	1	0.0	3281	5
$p18$	35	143	3228	0	3228	0	247	3	3228	3228	0	0.0	3228	10
$p19$	36	156	3710	0	3710	0	229	6	3710	3710	0	0.0	3710	14
$p20$	37	142	1830	1	1830	1	251	1	1830	1830	1	0.0	1860	20
$p21$	38	155	3660	0	3660	0	168	1	3660	3660	0	0.0	3660	18
$p22$	38	154	1912	0	1912	0	290	1	1912	1912	0	0.0	1912	20
$p23$	44	204	3770	0	3770	0	394	3	3770	3770	1	0.0	3810	21
$p24$	34	104	661	0	661	0	183	1	661	661	0	0.0	661	27
$p25$	36	120	504	0	504	0	265	1	504	504	0	0.0	504	23
$p26$	37	131	520	0	520	0	208	3	520	520	0	0.0	520	28
$p27$	44	174	216	1	216	2	730	12	216	216	0	0.0	216	7
$p28$	44	164	1729	0	1729	0	306	2	1729	1729	0	0.0	1729	44
$p29$	53	254	3470	0	3470	0	165	1	3470	3470	0	0.0	3470	65
$p30$	60	317	4891	0	4891	1	617	19	4891	4891	2	0.0	4891	56
$p31$	47	179	620	0	620	0	144	2	620	620	0	0.0	620	70
$p32$	51	211	2480	0	2480	0	342	1	2480	2480	0	0.0	2480	70
$p33$	56	258	3018	0	3018	1	672	11	3018	3018	0	0.0	3018	62
$p34$	74	421	1980	0	1980	2	1072	23	1980	1980	0	0.0	1980	131
$p35$	86	566	2140	0	2140	2	1204	24	2140	2140	0	0.0	2140	135
$p36$	101	798	7210	1	7210	4	1369	48	7210	7210	0	0.0	7210	163
$p38$	94	537	2130	1	2130	10	1893	29	2130	2130	0	0.0	2130	70
$p40$	86	497	4984	1	4984	13	2086	30	4879	4984	0	2.2	4984	224
$p41$	116	900	2688	4	2688	31	2524	41	2688	2688	0	0.0	2688	313
$p42$	138	1186	2466	60	2466	171	4445	34	2466	2509	2	1.7	2480	405

In Table 1 (WVCP instances) we report the instance name, the number of nodes n and the number of edges m in the first part. Then we give the results obtained by the new branch-and-price algorithm: the root node lower bound bd and the corresponding computing time, the value of the best solution found z (in bold when optimal) and the corresponding computing time, the optimality gap computed as $100(z - bd)/bd$, the number of generated columns ($cols$) and the number of explored nodes. All the computing times are in seconds; when the time limit of 1 h is reached, the table reports tl . The last part of the table reports the results obtained by two heuristic algorithms. For the set-covering heuristic proposed by Malaguti, Monaci and Toth and denoted as M–M–T [10], we report the lower bound bd computed in [10] by solving a compact ILP model, the solution value z and the corresponding computing time (in bold when optimal according to the lower bound internally computed by the algorithm) and the optimality gap (with respect to the same lower bound). The results were obtained on a PIV at 2.4 GHz with 512 Mb RAM under Windows XP operating system, which spent 7 s *user time* to solve the benchmark instance. For the variable neighborhood descent heuristic proposed by Oliveira, Noronha, and Urrutia and denoted as O–N–U [12], we report the solution value z and the corresponding computing time. The results were obtained on a PIV at 3.0 GHz with 1Gb RAM under the Linux operating system, which spent 8 s *user time* to solve the benchmark instance. Thus, we can conclude that the computer used in our experiments is approximately 1.3 times faster than the ones used to test the two heuristic algorithms M–M–T and O–N–U. In the tables, we report 0 s for computing times smaller than 1 s.

The new branch-and-price algorithm can solve 36 instances to proven optimality in no more than 22 s, except two instances where it took 447 and 2136 s. For the remaining 10 instances it runs into the time limit. For 9 of these instances, the comparison with the best solution reported for M–M–T and O–N–U certifies that the algorithm had not yet found the optimal solution. As a comparison, M–M–T, which is a heuristic algorithm, could solve to optimality 28 instances, but the lower bound computed at the beginning of the computation can certify optimality of only 22 solutions. The computing time of this algorithm never exceeds 182 s, with the exception of one instance which takes 237 s. Even though we do not make use of any primal heuristic in our exact approach, the branch-and-price algorithm improves 10 times on the best solution found by M–M–T (for one of these instances it runs out of time, while M–M–T takes 179 s to find a solution larger by one unit), and it finds worse solutions in 7 cases. The branch-and-price algorithm improves 11 times on the best solution found by O–N–U, while it finds worse solutions in 9 cases. However, the branch-and-price algorithm requires more computational effort, and runs into the time limit 10 times, while O–N–U always needs no more than 105 s of computing time, except

Table 3
Matrix decomposition R instances.

Instance	n	m	B&P					P-R [9]		
			bd	Time	z	Time	Cols	Nodes	z	Time
R01	144	1280	6724	13	6724	171	6,048	55	6724	196
R02	142	1246	6771	14	6771	200	6,387	63	6771	287
R03	139	1188	6473	84	6473	250	4,148	9	6475	275
R04	151	1406	6342	242	6342	565	5,928	23	6342	285
R05	142	1266	6408	110	6408	458	6,613	24	6435	254
R06	148	1381	7550	11	7550	136	5,434	83	7550	280
R07	141	1253	6889	16	6889	210	6,023	62	6889	275
R08	138	1191	6057	33	6057	289	6,565	40	6075	273
R09	129	1027	6358	53	6358	241	5,039	24	6429	266
R10	150	1409	6508	35	6508	227	8,265	74	6525	293
R11	208	2247	7654	2340	7654	2973	9,909	8	7681	145
R12	199	2055	7690	323	7690	3158	17,111	86	7696	569
R13	217	2449	7500	2480	7500	6461	20,737	20	7530	660
R14	214	2387	8254	166	8254	1420	12,577	137	8254	665
R15	198	2055	8021	169	8021	1157	8,907	88	8021	685
R16	188	1861	7755	78	7755	712	9,563	77	7755	688
R17	213	2392	7979	217	7979	2257	16,594	111	8033	709
R18	200	2079	7232	466	7232	2502	15,554	79	7257	218
R19	185	1803	6826	142	6826	1349	9,739	42	6866	365
R20	217	2447	8023	993	8023	993	6,801	1	8027	712
R21	281	3554	9284	7873	–	tl	22,049	57	9287	1749
R22	285	3684	8887	6258	8887	12,373	17,652	13	8888	1828
R23	288	3732	9136	8647	–	tl	15,327	9	9145	1970
R24	269	3284	8464	1739	–	tl	18,178	17	8464	1627
R25	266	3177	8355	tl	–	tl	10,441	1	8502	1586
R26	284	3629	8819	7533	–	tl	16,915	15	8840	1751
R27	259	3019	7975	465	7975	4396	17,044	108	7980	1459
R28	288	3765	9407	189	9407	3716	20,689	143	9407	2032
R29	281	3553	8693	1034	8693	10,604	28,551	167	8693	1535
R30	301	4122	9816	3527	–	tl	25,496	69	9816	2032

one instance where it takes 145 s. In addition, O–N–U is the only algorithm able to find the optimal solution of value 68 for instance GEOM110, while the branch-and-price algorithm runs into the time limit with a best incumbent solution of value 69.

As far as the lower bound provided by the linear relaxation of the set covering model (1)–(3) is concerned, it improves 22 times on the lower bound provided by the linear relaxation of the model proposed in [10] and used by M–M–T.

In Tables 2 and 3, we report similar information for two groups of matrix decomposition instances (transformed to WVCP instances), denoted as p and R , respectively. The last part of Table 2 reports the results obtained by M–M–T and by the GRASP heuristic by Prais and Ribeiro and denoted as P–R [9], while the last part of Table 3 reports the results obtained by P–R. The algorithm by P–R was run on an IBM 9672 model R34; Malaguti et al. [10] estimate this computer to be from 40 to 60 times slower than the one they used, and thus we estimate it to be from 50 to 70 times slower than the one we used in our experiments. The branch-and-price algorithm can solve all the “easy” p instances, thus improving on one solution and proving optimality of two additional instances with respect to M–M–T (which already improved on P–R). The computing times of the two algorithms are comparable for all but the last 4 instances, where the branch-and-price algorithm requires from 10 to 171 s, compared to at most 2 s of M–M–T.

When considering the R instances from [9], which are larger, we allow 4 h of computing time. The only comparison for these instances is with P–R, since in [10] only aggregated results are reported. The branch-and-price algorithm can solve to proven optimality all the 24 instances for which an integer solution is produced; for 13 of these instances, the solution value improves on the heuristic solution of P–R. However, the branch-and-price algorithm requires a substantial computational effort when compared with P–R, which always produces a feasible solution in at most 2032 s of computing time on a computer which is from 50 to 70 times slower. For the remaining 6 instances, in 5 cases the algorithm runs into time limit before reaching a leaf of the branching tree, and no integer solution is produced. In the remaining case (instance R25), the time limit is reached before the root node is optimally solved. In this case we report in Table 3 the Lagrangian lower bound when the computation is stopped. Instances in the R set do not appear structurally difficult, and the value of the computed lower bound always equals the value of the optimal solution when this is known. The computational difficulty comes from the size of these instances, which ask for the generation of a very large number of columns.

To conclude, the experiments confirm the effectiveness of the adopted rule for node selection in branching (that is, collapsing “heavy” nodes, as explained in Section 2.2). On the three considered sets of instances, the chosen strategy produces 8 better feasible solutions, and only 3 worse solutions, when compared with a random selection of the nodes to be collapsed.

4. Conclusions

We have considered the Weighted Vertex Coloring Problem, an NP-hard generalization of the Vertex Coloring which recently received large computational attention [9,10,12]. We proposed the first exact algorithm for the problem, based on column generation and branch-and-price. For branching, we considered a classical rule from the Vertex Coloring literature, that we have adapted to the specific weighted case so as to better exploit the problem structure. According to extensive computational experiments on three sets of instances from the literature, the proposed exact algorithm has an excellent performance even when compared with the best heuristic algorithms from the literature. Several new optimal solutions were obtained, and the best known lower bound for many others was improved by the approach described in this paper.

Acknowledgments

The authors would like to thank Stephan Held who kindly provided the code for the MWISP described in [15]; Charlotte Mitchell, for help with English writing; and two anonymous referees, whose comments greatly improved the presentation of the paper. The authors are partially supported by MIUR, Italy.

References

- [1] M.R. Garey, D.S. Johnson, *Computers and Intractability: A Guide to the Theory of NP-Completeness*, in: DIMACS Series in Discrete Mathematics and Theoretical Computer Science, Freedman, New York, 1979.
- [2] E. Malaguti, The vertex coloring problem and its generalizations, *4OR* 7 (2009) 101–104.
- [3] E. Malaguti, P. Toth, A survey on vertex coloring problems, *International Transactions in Operational Research* (17) (2010) 1–34.
- [4] D. De Werra, M. Demange, J. Monnot, V.T. Paschos, Time slot scheduling of compatible jobs, *Journal of Scheduling* 10 (2007) 111–127.
- [5] B. Escoffier, J. Monnot, V.T. Paschos, Weighted coloring: further complexity and approximability results, *Information Processing Letters* 94 (2006) 98–103.
- [6] M. Boudhar, G. Finke, Scheduling on a batch machine with job compatibilities, *Belgian Journal of Operations Research, Statistics and Computer Science* 40 (2000) 874–885.
- [7] G. Finke, V. Jost, M. Queyranne, A. Sebö, Batch processing with interval graph compatibilities between tasks, *Cahiers du laboratoire Leibniz* 108 (2004).
- [8] C.C. Ribeiro, M. Minoux, M.C. Penna, An optimal column-generation-with-ranking algorithm for very large scale set partitioning problems in traffic assignment, *European Journal of Operational Research* 41 (1989) 232–239.
- [9] M. Prais, C.C. Ribeiro, Reactive GRASP: an application to a matrix decomposition problem in TDMA traffic assignment, *INFORMS Journal on Computing* 12 (2000) 164–176.
- [10] E. Malaguti, M. Monaci, P. Toth, Models and heuristic algorithms for a weighted vertex coloring problem, *Journal of Heuristics* 15 (2009) 503–526.
- [11] A. Caprara, M. Fischetti, P. Toth, A heuristic method for the set covering problem, *Operations Research* 47 (1999) 730–743.
- [12] C. Oliveira, F.T. Noronha, S. Urrutia, Heurística VND com backtracking para o problema de coloração de vértices com pesos, in: *Proceedings of the XLIII Simpósio Brasileiro de Pesquisa Operacional*, 2011.
- [13] A. Mehrotra, M.A. Trick, A column generation approach for graph coloring, *INFORMS Journal on Computing* 8 (1996) 344–354.
- [14] E. Malaguti, M. Monaci, P. Toth, An exact approach for the vertex coloring problem, *Discrete Optimization* 8 (2011) 174–190.
- [15] S. Held, W. Cook, E.C. Sewell, Safe lower bounds for graph coloring, in: *Proceedings of the 15th International IPCO Conference on Integer Programming and Combinatorial Optimization*, Springer-Verlag, 2011, pp. 261–273.
- [16] A.A. Zykov, On some properties of linear complexes, *Matematicheskij Sbornik* (24) (1949) 163–188.
- [17] M.A. Trick, *Computational Symposium: Graph Coloring and its Generalizations*, Cornell University, Ithaca, NY, 2002, <http://mat.gsia.cmu.edu/COLOR02/>.
- [18] D.S. Johnson, M.A. Trick (Eds.), *Cliques, Coloring, and Satisfiability: 2nd DIMACS Implementation Challenge*, 1993, in: DIMACS Series in Discrete Mathematics and Theoretical Computer Science, American Mathematical Society, 1996.